

Лабораторная работа №3. Генераторы псевдослучайных чисел

Создано системой Doxygen 1.17.0

Глава 1

Лабораторная работа №3. Исследование генераторов псевдослучайных чисел

1.1 Цель работы

Исследовать качество и скорость работы нескольких генераторов псевдослучайных чисел, сравнить их статистические характеристики и оценить пригодность для практического использования.

1.2 Постановка задачи

В работе реализованы и исследованы следующие генераторы псевдослучайных чисел:

- [LCG_BaysDurham](#) — линейный конгруэнтный генератор с перемешиванием Bays-Durham.
- [Xorshift128_MLT](#) — генератор на основе схемы xorshift128 с дополнительным смешиванием.
- [AdditiveLFG_RotateXor](#) — аддитивный лаговый рекуррентный генератор с циклическими сдвигами и XOR.

Для каждого генератора требуется:

1. Сформировать 20 выборок по 1000 чисел в диапазоне от 0 до 9999.
2. Вычислить описательные статистики: среднее значение, стандартное отклонение, коэффициент вариации и статистику хи-квадрат.
3. Проверить выборки на равномерность распределения по критерию хи-квадрат.
4. Выполнить набор NIST-подобных тестов для битовых последовательностей.
5. Измерить скорость генерации различного количества случайных чисел.
6. Сохранить результаты в CSV-файлы для построения графиков и включения в документацию.

1.3 Теоретические сведения

Псевдослучайные числа формируются детерминированными алгоритмами, которые по начальному зерну порождают последовательность, внешне похожую на случайную. Качество генератора оценивается по статистическим свойствам последовательности и по отсутствию заметных закономерностей в распределении значений и битов.

Для анализа в работе используются следующие характеристики:

- Среднее значение выборки.
- Стандартное отклонение.
- Коэффициент вариации.
- Статистика критерия хи-квадрат.
- p-value для NIST-подобных тестов.

Если значение статистики хи-квадрат не превышает критического уровня, гипотеза о равномерности распределения не отвергается. Для NIST-подобных тестов чем выше p-value, тем меньше оснований считать последовательность неслучайной.

1.4 Описание реализованных генераторов

1.4.1 LCG_BaysDurham

Генератор основан на линейном конгруэнтном методе. Для уменьшения корреляции между соседними значениями используется таблица перемешивания Bays-Durham из 32 элементов. После выбора значения из таблицы результат дополнительно проходит через этап смешивания битов.

1.4.2 Xorshift128_MLT

Генератор использует схему xorshift128, основанную на последовательности операций XOR и сдвигов. После получения очередного значения выполняется дополнительное нелинейное смешивание, что улучшает статистические свойства выходной последовательности.

1.4.3 AdditiveLFG_RotateXor

Данный генератор использует аддитивную лаговую рекуррентную схему с кольцевым буфером. Для повышения качества последовательности в вычислениях применяются циклические сдвиги, XOR и дополнительная функция перемешивания.

1.4.4 StandardMT

Для сравнения скорости работы дополнительно используется стандартный генератор `std::mt19937` ← `_64`, представляющий собой реализацию алгоритма Mersenne Twister.

1.5 Описание статистических тестов

1.5.1 Описательные статистики

Для каждой выборки вычисляются:

- среднее значение;
- стандартное отклонение;
- коэффициент вариации;
- статистика хи-квадрат;
- результат проверки на равномерность.

1.5.2 NIST-подобные тесты

В работе реализованы следующие тесты для битовой последовательности длиной 1000000 бит:

- Монобитный тест.
- Тест частоты в блоках.
- Тест серий.
- Тест на длину наибольшей серии единиц.
- Тест кумулятивных сумм в прямом направлении.
- Тест кумулятивных сумм в обратном направлении.

Эти тесты позволяют оценить баланс нулей и единиц, распределение единиц внутри блоков, структуру серий и наличие накопленных отклонений в последовательности.

1.6 Описание программы

Программа состоит из следующих основных частей:

- реализации трёх пользовательских генераторов;
- функций генерации выборок случайных чисел;
- функций вычисления описательных статистик;
- функций реализации NIST-подобных тестов;
- модуля измерения скорости генерации;
- блока сохранения результатов в файлы:
 - output/random_stats.csv;
 - output/nist_results.csv;
 - output/speed_results.csv.

1.7 Результаты экспериментов

1.7.1 Описательные статистики

В файл `random_stats.csv` сохраняются результаты для 20 выборок каждого генератора. По этим данным можно построить графики изменения:

- среднего значения;
- стандартного отклонения;
- коэффициента вариации;
- статистики хи-квадрат.

1.7.2 Графики описательных статистик

1.7.3 Результаты NIST-подобных тестов

Результаты тестов сохраняются в файл `nist_results.csv`. Для каждого генератора вычисляются `p-value` по всем реализованным тестам. Эти значения позволяют оценить случайность битовой последовательности и выявить возможные статистические отклонения.

1.7.4 График NIST-подобных тестов

1.7.5 Скорость генерации

Для измерения производительности программа вычисляет время генерации последовательностей длиной от 1000 до 1000000 значений. Результаты сохраняются в файл `speed_results.csv`.

1.7.6 График скорости генерации

1.8 Анализ результатов

По графику средних значений видно, что у всех трёх генераторов среднее по 20 выборкам колеблется около середины диапазона, то есть около 5000, без выраженного систематического смещения в большую или меньшую сторону. Это говорит о том, что на уровне выборочных средних все генераторы в целом дают приемлемое распределение значений в пределах исследуемого диапазона.

График стандартного отклонения показывает, что разброс значений у всех генераторов находится примерно на одном уровне, а колебания от выборки к выборке не выходят за разумные пределы. Коэффициент вариации также остаётся достаточно стабильным: у всех трёх генераторов он лежит примерно в интервале от 0.55 до 0.61, что указывает на близкую относительную изменчивость выборок.

По критерию хи-квадрат большинство выборок проходят проверку на равномерность, поскольку значения статистики обычно находятся ниже критического уровня 30.1435. Однако у каждого генератора есть отдельные выборки, в которых наблюдаются отклонения: у [LCG_BaysDurham](#) одна выборка имеет $(\chi^2) = 38.28$, у [Xorshift128_MLT](#) одна выборка достигает $(\chi^2) = 31.12$ и ещё

одна поднимается до 28.96, а у [AdditiveLFG_RotateXor](#) одна выборка даёт $(\chi^2) = 30.56$, то есть слегка превышает критический порог. В целом это означает, что все генераторы в основном демонстрируют равномерность, но полностью идеальным нельзя считать ни один из них.

Результаты NIST-подобных тестов показывают, что [LCG_BaysDurham](#) и [Xorshift128_MLT](#) проходят все использованные проверки по порогу $p\text{-value} = 0.01$, поскольку их значения остаются выше этой границы во всех тестах. Наиболее уверенно выглядит [Xorshift128_MLT](#): у него особенно высокое $p\text{-value}$ в тесте monobit (0.212764), тесте runs (0.729805) и на обратной кумулятивной сумме $p\text{-value}$ остаётся комфортно выше порога. У [LCG_BaysDurham](#) также хорошие результаты, причём особенно выделяется тест longest run с $p\text{-value}$ 0.950109.

Наиболее слабые результаты показывает [AdditiveLFG_RotateXor](#): хотя тесты monobit, block frequency, runs и longest run он проходит, значения $p\text{-value}$ для `cusum_forward` и `cusum_backward` равны 0.009880 и 0.007699 соответственно, то есть оказываются ниже контрольного порога 0.01. Это указывает на возможные накопленные отклонения в битовой последовательности и делает данный генератор менее устойчивым по сравнению с двумя остальными в рамках проведённых NIST-подобных проверок.

По графику скорости генерации видно, что при росте размера выборки время работы всех генераторов увеличивается почти линейно. Самым быстрым на больших объёмах данных оказывается [Xorshift128_MLT](#): при размере 1000000 значений он показывает 4.331 мс, тогда как [LCG_BaysDurham](#) требует 4.904 мс, [StandardMT](#) — 6.493 мс, а [AdditiveLFG_RotateXor](#) — 6.214 мс. На промежуточных объёмах [Xorshift128_MLT](#) также обычно остаётся среди лидеров, что делает его наиболее выгодным по соотношению качества и производительности.

Таким образом, по совокупности результатов наиболее удачным генератором в данной работе можно считать [Xorshift128_MLT](#), поскольку он показывает хорошие статистические характеристики, проходит все NIST-подобные тесты и демонстрирует лучшую скорость генерации на больших выборках. [LCG_BaysDurham](#) тоже даёт в целом качественные результаты, но уступает по скорости и имеет единичный заметный выброс по критерию хи-квадрат. [AdditiveLFG_RotateXor](#) показывает приемлемые описательные статистики, однако проигрывает по устойчивости в части NIST-подобных тестов и по производительности на больших объёмах генерации.

1.9 Выводы

В ходе лабораторной работы были реализованы и исследованы три генератора псевдослучайных чисел: [LCG_BaysDurham](#), [Xorshift128_MLT](#) и [AdditiveLFG_RotateXor](#).

В результате выполнения работы:

- получены выборки случайных чисел и вычислены их основные статистические характеристики;
- выполнена проверка равномерности по критерию хи-квадрат;
- проведены NIST-подобные тесты для битовых последовательностей;
- измерена скорость генерации и выполнено сравнение с `std::mt19937_64`.

Проведённое исследование показывает, что для оценки генераторов необходимо учитывать одновременно два аспекта: статистическое качество последовательности и производительность алгоритма. Наиболее предпочтительным можно считать тот генератор, который показывает устойчивые статистические характеристики, успешно проходит тесты случайности и при этом имеет высокую скорость генерации.

1.10 Репозиторий проекта

Исходный код лабораторной работы размещён в удалённом репозитории GitHub:

<https://github.com/KolbasovaTatiana/pseudo-random-numbers>

Глава 2

Алфавитный указатель классов

2.1 Классы

Классы с их кратким описанием.

AdditiveLFG_RotateXor	Аддитивный лаговый рекуррентный генератор с вращением и XOR	??
LCG_BaysDurham	Генератор на основе линейного конгруэнтного метода с перемешиванием Bays↔Durham	??
SampleStats	Структура статистик для одной выборки	??
StandardMT	Обёртка над стандартным генератором Mersenne Twister	??
Xorshift128_MLT	Генератор xorshift128 с дополнительным перемешиванием	??

Глава 3

Список файлов

3.1 Файлы

Полный список файлов.

[lab3_psrandom.cpp](#)

Лабораторная работа №3. Исследование генераторов псевдослучайных чисел . . . ??

Глава 4

Классы

4.1 Класс AdditiveLFG_RotateXor

Аддитивный лаговый рекуррентный генератор с вращением и XOR.

Открытые члены

- [AdditiveLFG_RotateXor](#) (uint32_t seed=123456789U)
Конструктор генератора.
- void [seed](#) (uint32_t seedValue)
Переинициализация генератора.
- uint32_t [next32](#) ()
Генерация 32-битного значения из буфера.
- uint32_t [next](#) ()
Генерация 32-битного значения.

4.1.1 Подробное описание

Аддитивный лаговый рекуррентный генератор с вращением и XOR.

Генератор использует кольцевой буфер, аддитивную схему с двумя лагами, циклические сдвиги и XOR-перемешивание.

См. определение в файле [lab3_psrandom.cpp](#) строка [176](#)

4.1.2 Конструктор(ы)

4.1.2.1 AdditiveLFG_RotateXor()

```
AdditiveLFG_RotateXor::AdditiveLFG_RotateXor (
    uint32_t seed = 123456789U) [inline], [explicit]
```

Конструктор генератора.

Аргументы

seed	Начальное зерно.
------	------------------

См. определение в файле [lab3_psrandom.cpp](#) строка [209](#)

4.1.3 Методы

4.1.3.1 next()

```
uint32_t AdditiveLFG_RotateXor::next () [inline]
```

Генерация 32-битного значения.

Возвращает

Следующее 32-битное значение.

См. определение в файле [lab3_psrandom.cpp](#) строка [234](#)

4.1.3.2 next32()

```
uint32_t AdditiveLFG_RotateXor::next32 () [inline]
```

Генерация 32-битного значения из буфера.

Возвращает

Следующее 32-битное значение.

См. определение в файле [lab3_psrandom.cpp](#) строка [227](#)

4.1.3.3 seed()

```
void AdditiveLFG_RotateXor::seed (
    uint32_t seedValue) [inline]
```

Переинициализация генератора.

Аргументы

seedValue	Новое зерно.
-----------	--------------

См. определение в файле [lab3_psrandom.cpp](#) строка 215

Объявления и описания членов класса находятся в файле:

- [lab3_psrandom.cpp](#)

4.2 Класс LCG_BaysDurham

Генератор на основе линейного конгруэнтного метода с перемешиванием Bays-Durham.

Открытые члены

- [LCG_BaysDurham](#) (uint64_t seed=123456789ULL)
Конструктор генератора.
- void [seed](#) (uint64_t seedValue)
Переинициализация генератора новым зерном.
- uint64_t [next64](#) ()
Генерация 64-битного псевдослучайного числа.
- uint32_t [next](#) ()
Генерация 32-битного числа.

4.2.1 Подробное описание

Генератор на основе линейного конгруэнтного метода с перемешиванием Bays-Durham.

Базовая последовательность получается линейным конгруэнтным генератором. Далее используется таблица перемешивания из 32 элементов для уменьшения корреляций.

См. определение в файле [lab3_psrandom.cpp](#) строка 60

4.2.2 Конструктор(ы)

4.2.2.1 LCG_BaysDurham()

```
LCG_BaysDurham::LCG_BaysDurham (
    uint64_t seed = 123456789ULL)  [inline], [explicit]
```

Конструктор генератора.

Аргументы

seed	Начальное зерно.
------	------------------

См. определение в файле [lab3_psrandom.cpp](#) строка 79

4.2.3 Методы

4.2.3.1 next()

```
uint32_t LCG_BaysDurham::next ()  [inline]
```

Генерация 32-битного числа.

Возвращает

Следующее 32-битное значение.

См. определение в файле [lab3_psrandom.cpp](#) строка 116

4.2.3.2 next64()

```
uint64_t LCG_BaysDurham::next64 ()  [inline]
```

Генерация 64-битного псевдослучайного числа.

Возвращает

Следующее 64-битное значение.

См. определение в файле [lab3_psrandom.cpp](#) строка 99

4.2.3.3 seed()

```
void LCG_BaysDurham::seed (  
    uint64_t seedValue) [inline]
```

Переинициализация генератора новым зерном.

Аргументы

seedValue	Новое зерно.
-----------	--------------

См. определение в файле [lab3_psrandom.cpp](#) строка 91

Объявления и описания членов класса находятся в файле:

- [lab3_psrandom.cpp](#)

4.3 Структура SampleStats

Структура статистик для одной выборки.

Открытые атрибуты

- double [mean](#)
Среднее значение.
- double [stddev](#)
Стандартное отклонение.
- double [cv](#)
Коэффициент вариации.
- double [chi2](#)
Статистика хи-квадрат.
- bool [uniform](#)
Результат проверки на равномерность.

4.3.1 Подробное описание

Структура статистик для одной выборки.

См. определение в файле [lab3_psrandom.cpp](#) строка 240

4.3.2 Данные класса

4.3.2.1 chi2

```
double SampleStats::chi2
```

Статистика хи-квадрат.

См. определение в файле [lab3_psrandom.cpp](#) строка 244

4.3.2.2 cv

`double SampleStats::cv`

Коэффициент вариации.

См. определение в файле [lab3_psrandom.cpp](#) строка 243

4.3.2.3 mean

`double SampleStats::mean`

Среднее значение.

См. определение в файле [lab3_psrandom.cpp](#) строка 241

4.3.2.4 stddev

`double SampleStats::stddev`

Стандартное отклонение.

См. определение в файле [lab3_psrandom.cpp](#) строка 242

4.3.2.5 uniform

`bool SampleStats::uniform`

Результат проверки на равномерность.

См. определение в файле [lab3_psrandom.cpp](#) строка 245

Объявления и описания членов структуры находятся в файле:

- [lab3_psrandom.cpp](#)

4.4 Класс StandardMT

Обёртка над стандартным генератором Mersenne Twister.

Открытые члены

- [StandardMT](#) (uint64_t seed=12345)
Конструктор генератора.
- void [seed](#) (uint64_t seedValue)
Переинициализация генератора.
- uint32_t [next](#) ()
Генерация 32-битного значения.

4.4.1 Подробное описание

Обёртка над стандартным генератором Mersenne Twister.

Используется для сравнения скорости с пользовательскими генераторами.

См. определение в файле [lab3_psrandom.cpp](#) строка 437

4.4.2 Конструктор(ы)

4.4.2.1 StandardMT()

```
StandardMT::StandardMT (  
    uint64_t seed = 12345) [inline], [explicit]
```

Конструктор генератора.

Аргументы

seed	Начальное зерно.
------	------------------

См. определение в файле [lab3_psrandom.cpp](#) строка 443

4.4.3 Методы

4.4.3.1 next()

```
uint32_t StandardMT::next () [inline]
```

Генерация 32-битного значения.

Возвращает

Следующее псевдослучайное число.

См. определение в файле [lab3_psrandom.cpp](#) строка 453

4.4.3.2 seed()

```
void StandardMT::seed (  
    uint64_t seedValue) [inline]
```

Переинициализация генератора.

Аргументы

seedValue	Новое зерно.
-----------	--------------

См. определение в файле [lab3_psrandom.cpp](#) строка 449

Объявления и описания членов класса находятся в файле:

- [lab3_psrandom.cpp](#)

4.5 Класс Xorshift128_MLT

Генератор xorshift128 с дополнительным перемешиванием.

Открытые члены

- [Xorshift128_MLT](#) (uint64_t seed=123456789ULL)
Конструктор генератора.
- void [seed](#) (uint64_t seedValue)
Переинициализация генератора.
- uint64_t [next64](#) ()
Генерация 64-битного числа.
- uint32_t [next](#) ()
Генерация 32-битного числа.

4.5.1 Подробное описание

Генератор xorshift128 с дополнительным перемешиванием.

Используется схема xorshift128, после которой результат дополнительно подвергается смешиванию для улучшения статистических свойств.

См. определение в файле [lab3_psrandom.cpp](#) строка [125](#)

4.5.2 Конструктор(ы)

4.5.2.1 Xorshift128_MLT()

```
Xorshift128_MLT::Xorshift128_MLT (
    uint64_t seed = 123456789ULL) [inline], [explicit]
```

Конструктор генератора.

Аргументы

seed	Начальное зерно.
------	------------------

См. определение в файле [lab3_psrandom.cpp](#) строка [131](#)

4.5.3 Методы

4.5.3.1 next()

```
uint32_t Xorshift128_MLT::next () [inline]
```

Генерация 32-битного числа.

Возвращает

Следующее 32-битное значение.

См. определение в файле [lab3_psrandom.cpp](#) строка [167](#)

4.5.3.2 next64()

```
uint64_t Xorshift128_MLT::next64 () [inline]
```

Генерация 64-битного числа.

Возвращает

Следующее 64-битное значение.

См. определение в файле [lab3_psrandom.cpp](#) строка 146

4.5.3.3 seed()

```
void Xorshift128_MLT::seed (  
    uint64_t seedValue) [inline]
```

Переинициализация генератора.

Аргументы

seedValue	Новое зерно.
-----------	--------------

См. определение в файле [lab3_psrandom.cpp](#) строка 137

Объявления и описания членов класса находятся в файле:

- [lab3_psrandom.cpp](#)

Глава 5

Файлы

5.1 Файл lab3_psrandom.cpp

Лабораторная работа №3. Исследование генераторов псевдослучайных чисел.

```
#include <algorithm>
#include <array>
#include <chrono>
#include <cmath>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <functional>
#include <iomanip>
#include <iostream>
#include <limits>
#include <numeric>
#include <random>
#include <string>
#include <vector>
```

Классы

- class [LCG_BaysDurham](#)
Генератор на основе линейного конгруэнтного метода с перемешиванием Bays-Durham.
- class [Xorshift128_MLT](#)
Генератор xorshift128 с дополнительным перемешиванием.
- class [AdditiveLFG_RotateXor](#)
Аддитивный лаговый рекуррентный генератор с вращением и XOR.
- struct [SampleStats](#)
Структура статистик для одной выборки.
- class [StandardMT](#)
Обёртка над стандартным генератором Mersenne Twister.

Функции

- `uint32_t rotl32 (uint32_t x, unsigned r)`
Циклический сдвиг 32-битного числа влево.
- `SampleStats computeStats (const vector< int > &sample)`
Вычисляет статистики для выборки случайных чисел.
- `template<typename Gen>
vector< int > generateSample (Gen &gen, int size, int range)`
Генерирует выборку случайных чисел заданного размера.
- `double my_erfc (double x)`
Аппроксимация дополнительной функции ошибок.
- `double p_value_from_chi2 (double chi2, int df)`
Вычисляет p-value для статистики хи-квадрат.
- `double nist_monobit (const vector< uint8_t > &bits)`
NIST-подобный монобитный тест.
- `double nist_block_frequency (const vector< uint8_t > &bits, int M=128)`
NIST-подобный тест частоты в блоках.
- `double nist_runs (const vector< uint8_t > &bits)`
NIST-подобный тест серий.
- `double nist_longest_run (const vector< uint8_t > &bits)`
NIST-подобный тест на длину наибольшей серии единиц.
- `double nist_cusum (const vector< uint8_t > &bits, bool forward=true)`
NIST-подобный тест кумулятивных сумм.
- `template<typename Gen>
vector< uint8_t > generateBits (Gen &gen, int numBits)`
Генерирует битовую последовательность.
- `template<typename Gen>
double measureGenerationTime (Gen &gen, int count)`
Измеряет время генерации заданного количества чисел.
- `int main ()`
Точка входа в программу.

Переменные

- `const int NUM_SAMPLES = 20`
Количество выборок для статистического анализа.
- `const int SAMPLE_SIZE = 1000`
Размер одной выборки.
- `const int RANGE_SIZE = 10000`
Размер диапазона случайных чисел.
- `const int K = 20`
Количество интервалов для критерия хи-квадрат.
- `const int BIN_SIZE = RANGE_SIZE / K`
Размер одного интервала для критерия хи-квадрат.
- `const double CHI2_CRIT = 30.1435`
Критическое значение критерия хи-квадрат при 19 степенях свободы и $\alpha = 0.05$.
- `const int BIT_COUNT = 1000000`
Количество битов для NIST-подобных тестов.

5.1.1 Подробное описание

Лабораторная работа №3. Исследование генераторов псевдослучайных чисел.

Программа реализует три пользовательских генератора псевдослучайных чисел, сравнивает их статистические характеристики, выполняет упрощённые NIST-подобные тесты и измеряет скорость генерации. Результаты сохраняются в CSV-файлы для последующего построения графиков и включения в отчёт Doxygen.

См. определение в файле [lab3_psrandom.cpp](#)

5.1.2 Функции

5.1.2.1 computeStats()

```
SampleStats computeStats (  
    const vector< int > & sample)
```

Вычисляет статистики для выборки случайных чисел.

Аргументы

sample	Выборка целых чисел.
--------	----------------------

Возвращает

Заполненная структура статистик.

См. определение в файле [lab3_psrandom.cpp](#) строка [251](#)

5.1.2.2 generateBits()

```
template<typename Gen>  
vector< uint8_t > generateBits (  
    Gen & gen,  
    int numBits)
```

Генерирует битовую последовательность.

Параметры шаблона

Gen	Тип генератора.
-----	-----------------

Аргументы

gen	Экземпляр генератора.
numBits	Количество бит.

Возвращает

Вектор битов.

См. определение в файле [lab3_psrandom.cpp](#) строка [462](#)

5.1.2.3 generateSample()

```
template<typename Gen>
vector< int > generateSample (
    Gen & gen,
    int size,
    int range)
```

Генерирует выборку случайных чисел заданного размера.

Параметры шаблона

Gen	Тип генератора.
-----	-----------------

Аргументы

gen	Экземпляр генератора.
size	Размер выборки.
range	Размер диапазона значений.

Возвращает

Вектор случайных чисел.

См. определение в файле [lab3_psrandom.cpp](#) строка 287

5.1.2.4 main()

```
int main ()
```

Точка входа в программу.

Программа:

- вычисляет описательные статистики для 20 выборок каждого генератора;
- сохраняет их в файл output/random_stats.csv;
- выполняет NIST-подобные тесты и сохраняет результаты в output/nist_results.csv;
- измеряет скорость генерации и сохраняет её в output/speed_results.csv.

См. определение в файле [lab3_psrandom.cpp](#) строка 495

5.1.2.5 measureGenerationTime()

```
template<typename Gen>
double measureGenerationTime (
    Gen & gen,
    int count)
```

Измеряет время генерации заданного количества чисел.

Параметры шаблона

Gen	Тип генератора.
-----	-----------------

Аргументы

gen	Экземпляр генератора.
count	Количество генерируемых значений.

Возвращает

Время генерации в миллисекундах.

См. определение в файле [lab3_psrandom.cpp](#) строка [478](#)

5.1.2.6 my_erfc()

```
double my_erfc (
    double x)
```

Аппроксимация дополнительной функции ошибок.

Аргументы

x	Аргумент.
---	-----------

Возвращает

Значение $\text{erfc}(x)$.

См. определение в файле [lab3_psrandom.cpp](#) строка [298](#)

5.1.2.7 nist_block_frequency()

```
double nist_block_frequency (
    const vector< uint8_t > & bits,
    int M = 128)
```

NIST-подобный тест частоты в блоках.

Аргументы

bits	Последовательность битов.
M	Размер блока.

Возвращает

p-value теста.

См. определение в файле [lab3_psrandom.cpp](#) строка [331](#)

5.1.2.8 nist_cusum()

```
double nist_cusum (
    const vector< uint8_t > & bits,
    bool forward = true)
```

NIST-подобный тест кумулятивных сумм.

Аргументы

bits	Последовательность битов.
forward	Направление обхода последовательности.

Возвращает

p-value теста.

См. определение в файле [lab3_psrandom.cpp](#) строка [412](#)

5.1.2.9 nist_longest_run()

```
double nist_longest_run (
    const vector< uint8_t > & bits)
```

NIST-подобный тест на длину наибольшей серии единиц.

Аргументы

bits	Последовательность битов.
------	---------------------------

Возвращает

p-value теста.

См. определение в файле [lab3_psrandom.cpp](#) строка [369](#)

5.1.2.10 nist_monobit()

```
double nist_monobit (  
    const vector< uint8_t > & bits)
```

NIST-подобный монобитный тест.

Аргументы

bits	Последовательность битов.
------	---------------------------

Возвращает

p-value теста.

См. определение в файле [lab3_psrandom.cpp](#) строка [318](#)

5.1.2.11 nist_runs()

```
double nist_runs (  
    const vector< uint8_t > & bits)
```

NIST-подобный тест серий.

Аргументы

bits	Последовательность битов.
------	---------------------------

Возвращает

p-value теста.

См. определение в файле [lab3_psrandom.cpp](#) строка [350](#)

5.1.2.12 p_value_from_chi2()

```
double p_value_from_chi2 (
    double chi2,
    int df)
```

Вычисляет p-value для статистики хи-квадрат.

Аргументы

chi2	Значение статистики.
df	Число степеней свободы.

Возвращает

Приближённое p-value.

См. определение в файле [lab3_psrandom.cpp](#) строка 310

5.1.2.13 rotl32()

```
uint32_t rotl32 (
    uint32_t x,
    unsigned r) [inline]
```

Циклический сдвиг 32-битного числа влево.

Аргументы

x	Исходное значение.
r	Количество бит для сдвига.

Возвращает

Результат циклического сдвига.

См. определение в файле [lab3_psrandom.cpp](#) строка 52

5.1.3 Переменные

5.1.3.1 BIN_SIZE

```
const int BIN_SIZE = RANGE_SIZE / K
```

Размер одного интервала для критерия хи-квадрат.

См. определение в файле [lab3_psrandom.cpp](#) строка 40

5.1.3.2 BIT_COUNT

```
const int BIT_COUNT = 1000000
```

Количество битов для NIST-подобных тестов.

См. определение в файле `lab3_psrandom.cpp` строка [46](#)

5.1.3.3 CHI2_CRIT

```
const double CHI2_CRIT = 30.1435
```

Критическое значение критерия хи-квадрат при 19 степенях свободы и $\alpha = 0.05$.

См. определение в файле `lab3_psrandom.cpp` строка [43](#)

5.1.3.4 K

```
const int K = 20
```

Количество интервалов для критерия хи-квадрат.

См. определение в файле `lab3_psrandom.cpp` строка [37](#)

5.1.3.5 NUM_SAMPLES

```
const int NUM_SAMPLES = 20
```

Количество выборок для статистического анализа.

См. определение в файле `lab3_psrandom.cpp` строка [28](#)

5.1.3.6 RANGE_SIZE

```
const int RANGE_SIZE = 10000
```

Размер диапазона случайных чисел.

См. определение в файле `lab3_psrandom.cpp` строка [34](#)

5.1.3.7 SAMPLE_SIZE

```
const int SAMPLE_SIZE = 1000
```

Размер одной выборки.

См. определение в файле `lab3_psrandom.cpp` строка [31](#)

5.2 lab3_psrandom.cpp

См. документацию.

```

00001
00008
00009 #include <algorithm>
00010 #include <array>
00011 #include <chrono>
00012 #include <cmath>
00013 #include <cstdlib>
00014 #include <cstdio>
00015 #include <cstdlib>
00016 #include <functional>
00017 #include <iomanip>
00018 #include <iostream>
00019 #include <limits>
00020 #include <numeric>
00021 #include <random>
00022 #include <string>
00023 #include <vector>
00024
00025 using namespace std;
00026
00028 const int NUM_SAMPLES = 20;
00029
00031 const int SAMPLE_SIZE = 1000;
00032
00034 const int RANGE_SIZE = 10000;
00035
00037 const int K = 20;
00038
00040 const int BIN_SIZE = RANGE_SIZE / K;
00041
00043 const double CHI2_CRIT = 30.1435;
00044
00046 const int BIT_COUNT = 1000000;
00047
00052 inline uint32_t rotl32(uint32_t x, unsigned r) {
00053     return (x << r) | (x >> (32 - r));
00054 }
00055
00060 class LCG_BaysDurham {
00061     static const uint64_t A = 6364136223846793005ULL;
00062     static const uint64_t C = 1442695040888963407ULL;
00063     static const int table_size = 32;
00064
00065     uint64_t state;
00066     array<uint64_t, table_size> table{};
00067     uint64_t last;
00068
00071     uint64_t rawLCG() {
00072         state = A * state + C;
00073         return state;
00074     }
00075
00076 public:
00079     explicit LCG_BaysDurham(uint64_t seed = 123456789ULL) {
00080         if (seed != 0)
00081             state = seed;
00082         else
00083             state = 123456789ULL;
00084         last = 0;
00085         for (int i = 0; i < table_size; ++i) table[i] = rawLCG();
00086         last = rawLCG();
00087     }
00088
00091     void seed(uint64_t seedValue) {
00092         state = seedValue ? seedValue : 123456789ULL;
00093         for (int i = 0; i < table_size; ++i) table[i] = rawLCG();
00094         last = rawLCG();
00095     }
00096
00099     uint64_t next64() {
00100         uint64_t x = rawLCG();
00101         size_t j = static_cast<size_t>((last >> 59) & (table_size - 1));
00102         uint64_t y = table[j];
00103         table[j] = x;
00104         last = y;
00105
00106         y ^= (y >> 27);
00107         y *= 0x3C79AC492BA7B653ULL;
00108         y ^= (y >> 33);
00109         y *= 0x1C69B3F74AC4AE35ULL;
00110         y ^= (y >> 27);
00111         return y;

```



```

00112     }
00113
00116     uint32_t next() {
00117         return static_cast<uint32_t>(next64());
00118     }
00119 };
00120
00125 class Xorshift128_MLT {
00126     uint64_t s[2];
00127
00128 public:
00131     explicit Xorshift128_MLT(uint64_t seed = 123456789ULL) {
00132         this->seed(seed);
00133     }
00134
00137     void seed(uint64_t seedValue) {
00138         if (seedValue == 0) seedValue = 123456789ULL;
00139         s[0] = seedValue;
00140         s[1] = seedValue ^ 0x9e3779b97f4a7c15ULL;
00141         for (int i = 0; i < 16; ++i) next64();
00142     }
00143
00146     uint64_t next64() {
00147         uint64_t s1 = s[0];
00148         uint64_t s0 = s[1];
00149         s[0] = s0;
00150         s1 ^= s1 << 23;
00151         s1 ^= s1 >> 17;
00152         s1 ^= s0;
00153         s1 ^= s0 >> 26;
00154         s[1] = s1;
00155
00156         uint64_t result = s1 + s0;
00157         result ^= (result >> 30);
00158         result *= 0xbf58476d1ce4e5b9ULL;
00159         result ^= (result >> 27);
00160         result *= 0x94d049bb133111ebULL;
00161         result ^= (result >> 31);
00162         return result;
00163     }
00164
00167     uint32_t next() {
00168         return static_cast<uint32_t>(next64());
00169     }
00170 };
00171
00176 class AdditiveLFG_RotateXor {
00177     static const int LAG_A = 55;
00178     static const int LAG_B = 24;
00179
00180     array<uint32_t, LAG_A> buffer{};
00181     int index;
00182
00186     static uint32_t scramble(uint32_t x) {
00187         x = rotl32(x, 7) ^ (x >> 3);
00188         x += 0x9e3779b9U;
00189         x ^= rotl32(x, 11);
00190         x ^= (x >> 16);
00191         return x;
00192     }
00193
00195     void refill() {
00196         for (int i = 0; i < LAG_A; ++i) {
00197             int j = (i + LAG_B) % LAG_A;
00198             uint32_t v = buffer[i] + buffer[j];
00199             v ^= rotl32(buffer[j], 9);
00200             v = scramble(v);
00201             buffer[i] = v;
00202         }
00203         index = 0;
00204     }
00205
00206 public:
00209     explicit AdditiveLFG_RotateXor(uint32_t seed = 123456789U) {
00210         this->seed(seed);
00211     }
00212
00215     void seed(uint32_t seedValue) {
00216         uint32_t x = seedValue ? seedValue : 123456789U;
00217         for (int i = 0; i < LAG_A; ++i) {
00218             x = 1664525U * x + 1013904223U;
00219             buffer[i] = scramble(x + static_cast<uint32_t>(i * 2654435761U));
00220         }
00221         index = LAG_A;
00222         for (int k = 0; k < 8; ++k) refill();
00223     }
00224

```

```

00227     uint32_t next32() {
00228         if (index >= LAG_A) refill();
00229         return buffer[index++];
00230     }
00231
00232     uint32_t next() {
00233         return next32();
00234     }
00235 };
00236
00237 struct SampleStats {
00238     double mean;
00239     double stddev;
00240     double cv;
00241     double chi2;
00242     bool uniform;
00243 };
00244
00245 SampleStats computeStats(const vector<int>& sample) {
00246     int n = static_cast<int>(sample.size());
00247     double sum = 0.0;
00248     for (int x : sample) sum += x;
00249     double mean = sum / n;
00250
00251     double sq = 0.0;
00252     for (int x : sample) sq += (x - mean) * (x - mean);
00253     double stddev = sqrt(sq / (n - 1));
00254     double cv = (mean != 0.0) ? (stddev / mean) : 0.0;
00255
00256     int bins[K] = { 0 };
00257     for (int x : sample) {
00258         int bin = x / BIN_SIZE;
00259         if (bin >= K) bin = K - 1;
00260         ++bins[bin];
00261     }
00262
00263     double expected = double(n) / K;
00264     double chi2 = 0.0;
00265     for (int i = 0; i < K; ++i) {
00266         double diff = bins[i] - expected;
00267         chi2 += (diff * diff) / expected;
00268     }
00269
00270     bool uniform = (chi2 <= CHI2_CRIT);
00271     return { mean, stddev, cv, chi2, uniform };
00272 }
00273
00274 template<typename Gen>
00275 vector<int> generateSample(Gen& gen, int size, int range) {
00276     vector<int> sample;
00277     sample.reserve(size);
00278     for (int i = 0; i < size; ++i)
00279         sample.push_back(static_cast<int>(gen.next() % range));
00280     return sample;
00281 }
00282
00283 double my_erfc(double x) {
00284     double t = 1.0 / (1.0 + 0.5 * fabs(x));
00285     double tau = t * exp(-x * x - 1.26551223 + t * (1.00002368 + t * (0.37409196 +
00286         t * (0.09678418 + t * (-0.18628806 + t * (0.27886807 + t * (-1.13520398 +
00287         t * (1.48851587 + t * (-0.82215223 + t * 0.17087277)))))))));
00288     return (x >= 0) ? tau : 2.0 - tau;
00289 }
00290
00291 double p_value_from_chi2(double chi2, int df) {
00292     double z = (sqrt(2.0 * chi2) - sqrt(2.0 * df - 1.0));
00293     return my_erfc(fabs(z) / sqrt(2.0));
00294 }
00295
00296 double nist_monobit(const vector<uint8_t>& bits) {
00297     int n = static_cast<int>(bits.size());
00298     if (n < 100) return -1;
00299     int sum = 0;
00300     for (auto b : bits) sum += b ? 1 : -1;
00301     double s = sum / sqrt(static_cast<double>(n));
00302     return my_erfc(fabs(s) / sqrt(2.0));
00303 }
00304
00305 double nist_block_frequency(const vector<uint8_t>& bits, int M = 128) {
00306     int n = static_cast<int>(bits.size());
00307     if (n < M) return -1;
00308     int N = n / M;
00309     double chi2 = 0.0;
00310     for (int i = 0; i < N; ++i) {
00311         int ones = 0;
00312         for (int j = 0; j < M; ++j)
00313             if (bits[i * M + j]) ++ones;
00314     }

```

```

00340     double pi = static_cast<double>(ones) / M;
00341     chi2 += (pi - 0.5) * (pi - 0.5);
00342 }
00343 chi2 *= 4.0 * M;
00344 return p_value_from_chi2(chi2, N);
00345 }
00346
00350 double nist_runs(const vector<uint8_t>& bits) {
00351     int n = static_cast<int>(bits.size());
00352     if (n < 100) return -1;
00353     int ones = 0;
00354     for (auto b : bits) ones += b;
00355     double pi = static_cast<double>(ones) / n;
00356     if (fabs(pi - 0.5) >= 2.0 / sqrt(static_cast<double>(n))) return 0.0;
00357     int runs = 1;
00358     for (int i = 1; i < n; ++i)
00359         if (bits[i] != bits[i - 1]) ++runs;
00360     double num = runs - 2.0 * n * pi * (1.0 - pi);
00361     double den = 2.0 * sqrt(2.0 * n) * pi * (1.0 - pi);
00362     double z = num / den;
00363     return my_erfc(fabs(z) / sqrt(2.0));
00364 }
00365
00369 double nist_longest_run(const vector<uint8_t>& bits) {
00370     int n = static_cast<int>(bits.size());
00371     if (n < 128) return -1;
00372
00373     const int M = 10000;
00374     int N = n / M;
00375     if (N == 0) return -1;
00376     const int CAT = 7;
00377     vector<int> v(CAT, 0);
00378
00379     for (int i = 0; i < N; ++i) {
00380         int longest = 0, current = 0;
00381         for (int j = 0; j < M; ++j) {
00382             if (bits[i * M + j]) ++current;
00383             else {
00384                 if (current > longest) longest = current;
00385                 current = 0;
00386             }
00387         }
00388         if (current > longest) longest = current;
00389
00390         if (longest <= 10) v[0]++;
00391         else if (longest == 11) v[1]++;
00392         else if (longest == 12) v[2]++;
00393         else if (longest == 13) v[3]++;
00394         else if (longest == 14) v[4]++;
00395         else if (longest == 15) v[5]++;
00396         else v[6]++;
00397     }
00398
00399     double pi[CAT] = { 0.0882, 0.2092, 0.2483, 0.1933, 0.1208, 0.0675, 0.0727 };
00400     double chi2 = 0.0;
00401     for (int i = 0; i < CAT; ++i) {
00402         double expected = N * pi[i];
00403         chi2 += (v[i] - expected) * (v[i] - expected) / expected;
00404     }
00405     return p_value_from_chi2(chi2, CAT - 1);
00406 }
00407
00412 double nist_cusum(const vector<uint8_t>& bits, bool forward = true) {
00413     int n = static_cast<int>(bits.size());
00414     if (n < 100) return -1;
00415     vector<int> X(n, 0);
00416     int sum = 0;
00417     if (forward) {
00418         for (int i = 0; i < n; ++i) {
00419             sum += bits[i] ? 1 : -1;
00420             X[i] = sum;
00421         }
00422     }
00423     else {
00424         for (int i = n - 1; i >= 0; --i) {
00425             sum += bits[i] ? 1 : -1;
00426             X[n - 1 - i] = sum;
00427         }
00428     }
00429     int maxZ = 0;
00430     for (auto z : X) if (abs(z) > maxZ) maxZ = abs(z);
00431     double z_stat = maxZ / sqrt(static_cast<double>(n));
00432     return my_erfc(z_stat / sqrt(2.0));
00433 }
00434
00437 class StandardMT {
00438     mt19937_64 gen;

```

```

00439     uniform_int_distribution<uint32_t> dist;
00440 public:
00443     explicit StandardMT(uint64_t seed = 12345)
00444         : gen(seed), dist(0, numeric_limits<uint32_t>::max()) {
00445     }
00446
00449     void seed(uint64_t seedValue) { gen.seed(seedValue); }
00450
00453     uint32_t next() { return dist(gen); }
00454 };
00455
00461 template<typename Gen>
00462 vector<uint8_t> generateBits(Gen& gen, int numBits) {
00463     vector<uint8_t> bits;
00464     bits.reserve(numBits);
00465     for (int i = 0; i < numBits; ++i) {
00466         uint32_t r = gen.next();
00467         bits.push_back((r >> 31) & 1U);
00468     }
00469     return bits;
00470 }
00471
00477 template<typename Gen>
00478 double measureGenerationTime(Gen& gen, int count) {
00479     auto t0 = chrono::high_resolution_clock::now();
00480     for (int i = 0; i < count; ++i) {
00481         volatile uint32_t sink = gen.next();
00482         (void)sink;
00483     }
00484     auto t1 = chrono::high_resolution_clock::now();
00485     return chrono::duration<double, milli>(t1 - t0).count();
00486 }
00487
00495 int main() {
00496     auto now = chrono::high_resolution_clock::now().time_since_epoch().count();
00497     uint32_t baseSeed = static_cast<uint32_t>(now);
00498
00499     FILE* f = fopen("output/random_stats.csv", "w");
00500     fprintf(f, "generator,sample,mean,stddev,cv,chi2,uniform\n");
00501
00502     printf("LCG_BaysDurham results:\n");
00503     for (int s = 0; s < NUM_SAMPLES; ++s) {
00504         LCG_BaysDurham gen(baseSeed + s * 1000);
00505         auto sample = generateSample(gen, SAMPLE_SIZE, RANGE_SIZE);
00506         auto stats = computeStats(sample);
00507         printf("Sample %2d: mean=%7.1f stddev=%7.1f cv=%6.4f chi2=%7.3f %s\n",
00508             s + 1, stats.mean, stats.stddev, stats.cv, stats.chi2,
00509             stats.uniform ? "PASS" : "FAIL");
00510         fprintf(f, "LCG_BaysDurham,%d,%1f,%1f,%4f,%3f,%d\n",
00511             s + 1, stats.mean, stats.stddev, stats.cv, stats.chi2, stats.uniform);
00512     }
00513
00514     printf("\nXorshift128_MLT results:\n");
00515     for (int s = 0; s < NUM_SAMPLES; ++s) {
00516         Xorshift128_MLT gen(baseSeed + s * 2000);
00517         auto sample = generateSample(gen, SAMPLE_SIZE, RANGE_SIZE);
00518         auto stats = computeStats(sample);
00519         printf("Sample %2d: mean=%7.1f stddev=%7.1f cv=%6.4f chi2=%7.3f %s\n",
00520             s + 1, stats.mean, stats.stddev, stats.cv, stats.chi2,
00521             stats.uniform ? "PASS" : "FAIL");
00522         fprintf(f, "Xorshift128_MLT,%d,%1f,%1f,%4f,%3f,%d\n",
00523             s + 1, stats.mean, stats.stddev, stats.cv, stats.chi2, stats.uniform);
00524     }
00525
00526     printf("\nAdditiveLFG_RotateXor results:\n");
00527     for (int s = 0; s < NUM_SAMPLES; ++s) {
00528         AdditiveLFG_RotateXor gen(baseSeed + s * 3000);
00529         auto sample = generateSample(gen, SAMPLE_SIZE, RANGE_SIZE);
00530         auto stats = computeStats(sample);
00531         printf("Sample %2d: mean=%7.1f stddev=%7.1f cv=%6.4f chi2=%7.3f %s\n",
00532             s + 1, stats.mean, stats.stddev, stats.cv, stats.chi2,
00533             stats.uniform ? "PASS" : "FAIL");
00534         fprintf(f, "AdditiveLFG_RotateXor,%d,%1f,%1f,%4f,%3f,%d\n",
00535             s + 1, stats.mean, stats.stddev, stats.cv, stats.chi2, stats.uniform);
00536     }
00537     fclose(f);
00538     printf("\nDescriptive statistics saved to output/random_stats.csv\n");
00539
00540     FILE* nist_f = fopen("output/nist_results.csv", "w");
00541     fprintf(nist_f, "generator,monobit,block_freq,runs,longest_run,cusum_forward,cusum_backward\n");
00542
00543     vector<string> generators = { "LCG_BaysDurham", "Xorshift128_MLT", "AdditiveLFG_RotateXor" };
00544     for (int g = 0; g < 3; ++g) {
00545         vector<uint8_t> bits;
00546         uint32_t seed = baseSeed + g * 5000;
00547         if (g == 0) {
00548             LCG_BaysDurham gen(seed);

```

```

00549     bits = generateBits(gen, BIT_COUNT);
00550 }
00551 else if (g == 1) {
00552     Xorshift128_MLT gen(seed);
00553     bits = generateBits(gen, BIT_COUNT);
00554 }
00555 else {
00556     AdditiveLFG_RotateXor gen(seed);
00557     bits = generateBits(gen, BIT_COUNT);
00558 }
00559
00560 double mono = nist_monobit(bits);
00561 double block = nist_block_frequency(bits, 128);
00562 double runs = nist_runs(bits);
00563 double longest = nist_longest_run(bits);
00564 double cusum_f = nist_cusum(bits, true);
00565 double cusum_b = nist_cusum(bits, false);
00566
00567 printf("\n%s NIST-like results (p-values):\n", generators[g].c_str());
00568 printf("Monobit:           %.6f\n", mono);
00569 printf("Block Frequency: %.6f\n", block);
00570 printf("Runs:                %.6f\n", runs);
00571 printf("Longest Run:         %.6f\n", longest);
00572 printf("CUSUM forward:      %.6f\n", cusum_f);
00573 printf("CUSUM backward:     %.6f\n", cusum_b);
00574
00575 fprintf(nist_f, "%s,%.6f,%.6f,%.6f,%.6f,%.6f,%.6f,%.6f\n",
00576         generators[g].c_str(), mono, block, runs, longest, cusum_f, cusum_b);
00577 }
00578 fclose(nist_f);
00579 printf("\nNIST-style results saved to output/nist_results.csv\n");
00580
00581 const vector<int> SIZES = { 1000, 2000, 5000, 10000, 20000, 50000, 100000, 200000, 500000, 1000000 };
00582 FILE* speed_f = fopen("output/speed_results.csv", "w");
00583 fprintf(speed_f,
00584 "size,LCG_BaysDurham_ms,Xorshift128_MLT_ms,AdditiveLFG_RotateXor_ms,StandardMT_ms\n");
00585
00586 printf("\n%10s%18s%18s%24s%16s\n", "Size", "LCG_BaysDurham", "Xorshift128_MLT",
00587 "AdditiveLFG_RotateXor", "StandardMT");
00588 for (int sz : SIZES) {
00589     double tLCG, tXor, tLFG, tStd;
00590     {
00591         LCG_BaysDurham gen(baseSeed);
00592         tLCG = measureGenerationTime(gen, sz);
00593     }
00594     {
00595         Xorshift128_MLT gen(baseSeed + 1);
00596         tXor = measureGenerationTime(gen, sz);
00597     }
00598     {
00599         AdditiveLFG_RotateXor gen(baseSeed + 2);
00600         tLFG = measureGenerationTime(gen, sz);
00601     }
00602     {
00603         StandardMT gen(baseSeed + 3);
00604         tStd = measureGenerationTime(gen, sz);
00605     }
00606     printf("%10d%18.3f%18.3f%24.3f%16.3f\n", sz, tLCG, tXor, tLFG, tStd);
00607     fprintf(speed_f, "%d,%.3f,%.3f,%.3f,%.3f\n", sz, tLCG, tXor, tLFG, tStd);
00608 }
00609 fclose(speed_f);
00610 printf("\nSpeed results saved to output/speed_results.csv\n");
00611 return 0;
00612 }

```

5.3 Файл README.md

